

Multi-Head Attention

Many questions, asked at the same time · video 2:00

This lesson has one big idea: instead of one big attention, run **several smaller ones side by side**, called **heads**. Each head has its own query, key and value matrices, so each can learn to look for something different. Their results are glued back together, mixed by one more learned matrix, and added to the token's vector.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 9).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. Several heads, side by side	▶ 0:08–0:29	3	☐
2. Sharing out the vector	▶ 0:29–0:40	4	☐
3. Different heads, different patterns	▶ 0:40–1:10	3	☐
4. Glue, mix and add	▶ 1:10–1:27	4	☐
5. Many views for the price of one	▶ 1:27–1:37	4	☐
6. Multi-head attention in code	▶ 1:37–1:57	3	☐

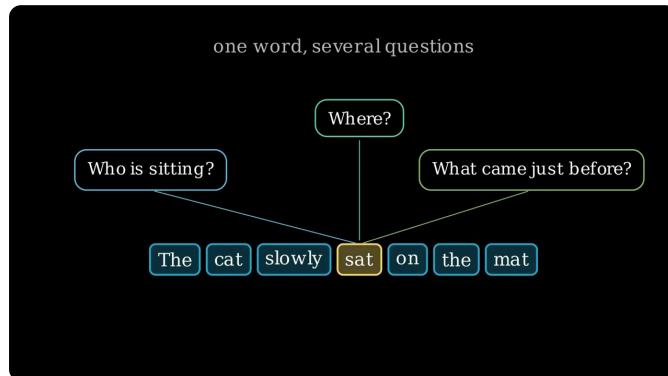
Before you start: this lesson builds directly on episodes 5 and 6: one head is exactly the attention of episode 6. It uses matrix shapes and multiplication (Foundations F03), and the code works with NumPy arrays that have an extra “heads” axis (F11).

CONCEPT 1 OF 6

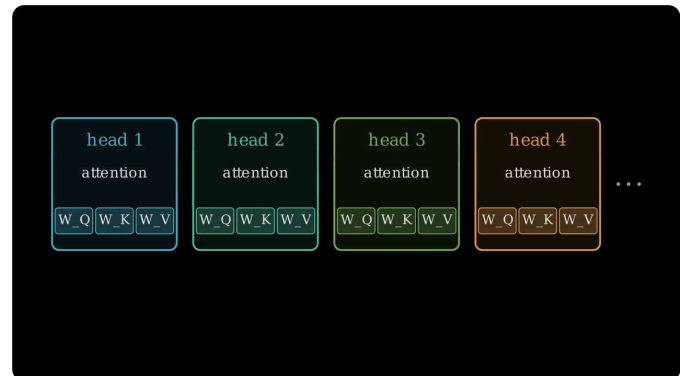
Several heads, side by side

▶ Video 0:08–0:29

One attention **head** (episode 6) asks one kind of question: each token gets one row of weights, so one blend of the others. But when the model reads *sat*, it might want to know several things at once. Who is sitting? Where? What came just before?



▶ 0:18 One word, several questions.



▶ 0:29 Several heads, each with its own W_Q , W_K , W_V .

So instead of one big attention, we run several smaller ones **side by side**. Each one is called a **head**, and each has its **own query, key and value matrices**.

KEY IDEA

Multi-head attention runs several attention heads in parallel, each with its own learned W_Q , W_K and W_V , so a token can ask several questions at once.

Check 1: Several heads, side by side

Answer all 3 questions before looking at the answers. Pass mark: all correct.

1.1 MULTIPLE CHOICE

What does each head have its own copy of?

- A. The token vectors X
- B. The matrices W_Q , W_K and W_V
- C. The vocabulary of tokens
- D. The text, in a different order

1.2 TRUE OR FALSE

"With a single head, *sat* gets one row of attention weights, so it can blend in only one mix of the other tokens."

True ☐ False ☐

1.3 SHORT ANSWER

In "*The dog quickly ran home*", the token *ran* asks "who is running?" and "what came just before?". Which token should each question find?

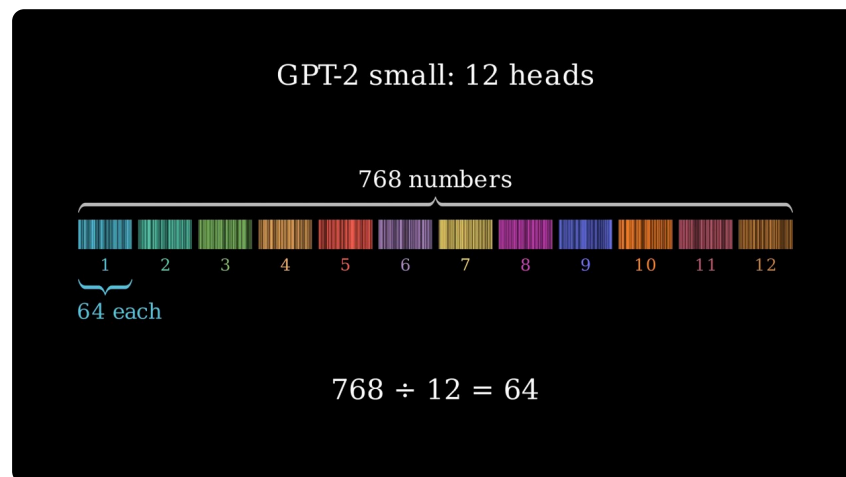
CONCEPT 2 OF 6

Sharing out the vector

▶ Video 0:29-0:40

Here's the trick: the heads **share out the vector**. GPT-2 small uses **12** heads on its **768** numbers, so each head works with **$768 \div 12 = 64$** of them.

Precisely: the whole token vector is projected to a query, a key and a value of 768 numbers each (as in episode 6), and each of those is cut into 12 pieces of 64, one piece per head. Every head then runs ordinary attention on its own 64-number pieces, scaling by $\sqrt{64} = 8$.



▶ 0:39 GPT-2 small: 12 heads share 768 numbers, 64 each.

KEY IDEA

head size = $d \div \text{number of heads}$ (GPT-2 small: $768 \div 12 = 64$). Each head gets its own 64-number query, key and value for every token.

Check 2: Sharing out the vector

Answer all 4 questions before looking at the answers. Pass mark: all correct.

2.1 CALCULATION

A model has vectors of 1,024 numbers and 16 heads. What is the head size?

.....

2.2 CALCULATION

A model has $d = 512$ and a head size of 64. How many heads does it have?

.....

2.3 TRUE OR FALSE

"Each GPT-2 small head only ever reads 64 of the token's original 768 numbers."

True ☐ False ☐

2.4 CALCULATION

For a 10-token text, how many attention scores does one head compute? And all 12 heads together?

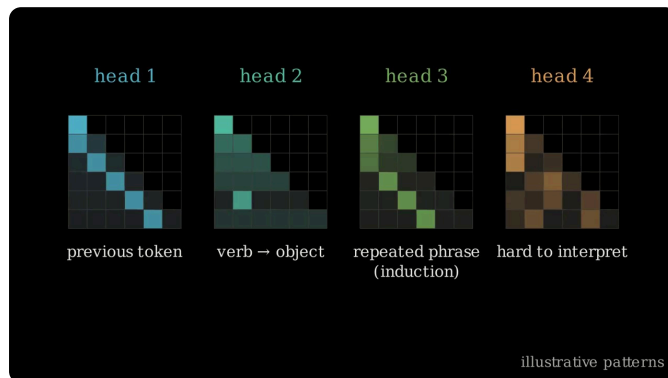
.....

CONCEPT 3 OF 6

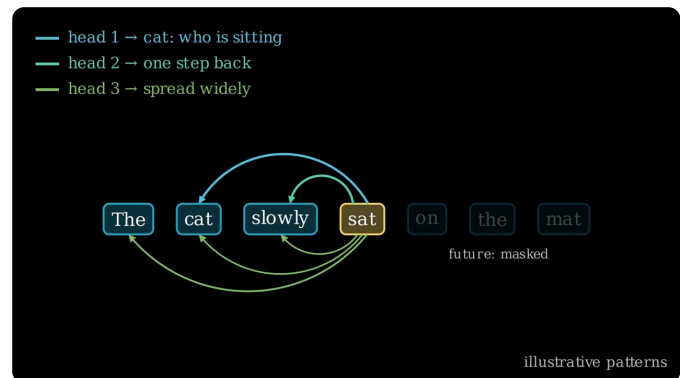
Different heads, different patterns

▶ Video 0:40–1:10

Because each head learns its **own matrices**, each can learn a **different pattern**. In trained models, researchers have found heads that look at the **previous token**, heads that connect a **verb to its object**, and heads that spot a **repeated phrase** and predict how it continues (**induction** heads). Many others are much harder to interpret. Nobody assigns these jobs: they come out of training.



▶ 0:58 Four heads, four patterns (row = token asking, column = token looked at).



▶ 1:10 On “sat”: three heads look in three different ways.

In the illustration, on *sat*, head 1 looks at *cat* (the one doing the sitting), head 2 looks one step back (*slowly*), and head 3 spreads its attention widely over the whole sentence.

KEY IDEA

Each head **learns its own attention pattern** from data. Some are easy to interpret (previous token, verb → object, induction); many are not.

Check 3: Different heads, different patterns

Answer all 3 questions before looking at the answers. Pass mark: all correct.

3.1 MULTIPLE CHOICE

In a “previous token” head's grid, where are the bright cells?

- A. On the diagonal: each token looks at itself
- B. Just left of the diagonal: row i , column $i - 1$
- C. All in the first column
- D. Above the diagonal

3.2 TRUE OR FALSE

“Engineers decide which head does what, for example ‘head 2 links verbs to their objects’.”

True ☐ False ☐

3.3 SHORT ANSWER

A text contains “... the Golden Gate Bridge ... the Golden Gate”. What would a repeated-phrase (induction) head help predict next, and why?

.....

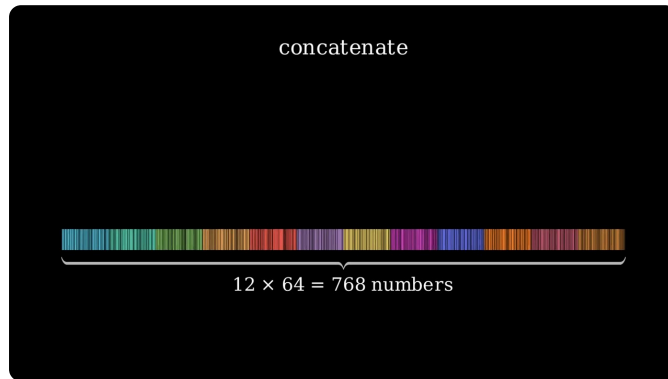
.....

CONCEPT 4 OF 6

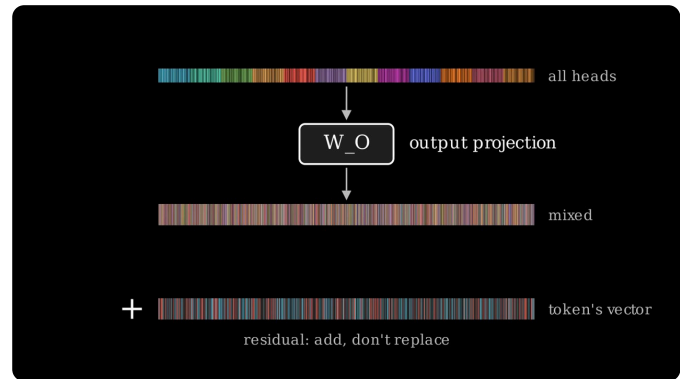
Glue, mix and add

▶ Video 1:10-1:27

Each head produces its own **64 numbers** for every token. We **glue them back together** (**concatenate**) into one vector of $12 \times 64 = \mathbf{768}$. Then one more learned matrix, the **output projection** $\mathbf{W}_O$, mixes what all the heads found, and the result is **added back to the token's vector**: added, not replaced, just like the weighted mix in episode 5.



▶ 1:18 Concatenate: $12 \times 64 = 768$ numbers.



▶ 1:26 $\mathbf{W}_O$ mixes the heads; the result is added to the token.

KEY IDEA

Concatenate the heads ($12 \times 64 = 768$) → multiply by $\mathbf{W}_O$ → **add** the result to the token's vector.

Check 4: Glue, mix and add

Answer all 4 questions before looking at the answers. Pass mark: all correct.

4.1 PUT IN ORDER

Put in order: *concatenate the heads · every head runs attention · add to the token's vector · multiply by $\mathbf{W}_O$ · project and split into heads.*

4.2 CALCULATION

In GPT-2 small, $\mathbf{W}_O$ takes 768 numbers to 768 numbers. How many learned numbers does it hold?

4.3 MULTIPLE CHOICE

What is $\mathbf{W}_O$ for?

- A.** It mixes what the different heads found into one result
- B.** It applies the causal mask
- C.** It turns the scores into weights that add up to 1
- D.** It splits the vector into heads

4.4 TRUE OR FALSE

"After multi-head attention, the token's vector is replaced by the result."

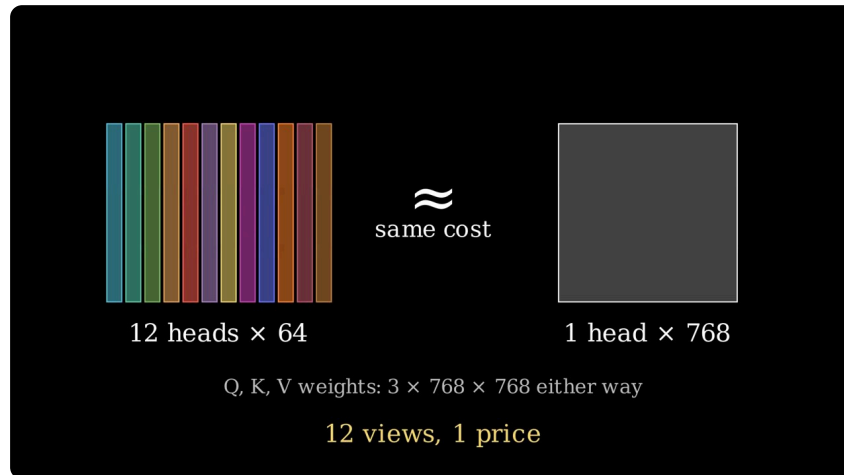
True ☐ False ☐

CONCEPT 5 OF 6

Many views for the price of one

▶ Video 1:27-1:37

The nice part: **twelve heads of size 64 cost about the same as one head of size 768**. Either way, the query, key and value matrices hold $3 \times 768 \times 768$ numbers: a full 768×768 W_Q is simply the twelve heads' 768×64 slices side by side. We get many points of view, for the price of one.



▶ 1:36 12 heads of 64 cost about the same as 1 head of 768: many views, one price.

KEY IDEA

Splitting into heads adds **no extra weights**: $12 \times (768 \times 64) = 768 \times 768$. Twelve attention patterns for the price of one.

Check 5: Many views for the price of one

Answer all 4 questions before looking at the answers. Pass mark: all correct.

5.1 CALCULATION

How many numbers are in one GPT-2 small layer's W_Q , W_K and W_V together?

5.2 CALCULATION

One head's slice of W_Q is 768×64 . How many numbers is that? And for all 12 heads?

5.3 TRUE OR FALSE

"Using 24 heads instead of 12 (still with $d = 768$) would double the query, key and value weights."

True ☐ False ☐

5.4 MULTIPLE CHOICE

What do 12 heads of size 64 give you that one head of size 768 does not?

- A. Twelve separate attention patterns for every token
- B. Twelve times as many weights
- C. Longer token vectors
- D. The ability to look at future tokens

CONCEPT 6 OF 6

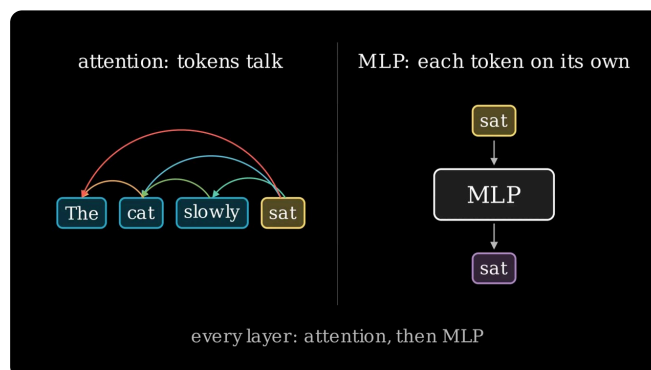
Multi-head attention in code

▶ Video 1:37-1:57

In code, we **split** the vectors into heads, run attention on **every head at once**, then **merge** the heads and apply the **output projection**. “Every head at once” works because the arrays get an extra first axis for the heads: `split` turns an (n, d) matrix into shape $(heads, n, hd)$, and the rest is episode 6's attention, done for all heads in one go.

```
def multi_head_attention(X, Wq, Wk, Wv, Wo, n_heads):
    n, d = X.shape
    hd = d // n_heads # 768 // 12 = 64
    def split(M):      # (n, d) -> (heads, n, hd)
        return M.reshape(n, n_heads, hd).transpose(1, 0, 2)
    Q, K, V = split(X @ Wq), split(X @ Wk), split(X @ Wv)
    scores = Q @ K.transpose(0, 2, 1) / np.sqrt(hd) # every head at once
    scores += np.triu(np.full((n, n), -np.inf), k=1) # causal mask
    w = np.exp(scores - scores.max(-1, keepdims=True))
    w /= w.sum(-1, keepdims=True) # softmax
    out = (w @ V).transpose(1, 0, 2).reshape(n, d) # merge the heads
    return out @ Wo # output projection
```

Attention lets tokens share information. Once a token has gathered its context, it needs to work with it on its own: that is the job of the other half of every layer, the **MLP** (episode 8).



▶ 1:55 Attention lets tokens talk; the MLP then works on each token on its own.

KEY IDEA

split $(n, d) \rightarrow (heads, n, hd)$, **attention** in every head at once, **merge** back to (n, d) , then **@** W_o .

Check 6: Multi-head attention in code

Answer all 3 questions before looking at the answers. Pass mark: all correct.

6.1 CALCULATION

GPT-2 small reads a 7-token text ($n = 7, d = 768, 12$ heads). What is the shape of (a) `Q` after `split`, (b) `scores`, (c) `out` before `@ Wo`?

6.2 TRUE OR FALSE

“In every layer, attention lets tokens share information, and the MLP then works on each token on its own.”

True ☐ False ☐

6.3 HANDS-ON CODE

Try it yourself. The code runs the video's `multi_head_attention` and episode 6's one-head `attention` on 3 tokens with $d = 4$, using a “do nothing” output projection ($W_O = \text{identity}$). (a) With `n_heads=2`, what are `hd` and the shapes of `Q` (after `split`) and `scores`? Add prints to check. (b) What do the three `print` lines show? (c) Explain the two True/False results: why is one head exactly episode 6's attention, and why do two heads give something different from the same weights?

```
import numpy as np

def attention(X, Wq, Wk, Wv):
    # episode 6: one head
    Q, K, V = X @ Wq, X @ Wk, X @ Wv
    scores = Q @ K.T / np.sqrt(K.shape[-1])
    n = len(X)
    scores[np.triu(np.ones((n, n)), k=1).astype(bool)] = -np.inf
    w = np.exp(scores - scores.max(-1, keepdims=True))
    w /= w.sum(-1, keepdims=True)
    return w @ V

def multi_head_attention(X, Wq, Wk, Wv, Wo, n_heads):
    n, d = X.shape
    hd = d // n_heads # 768 // 12 = 64
    def split(M):
        # (n, d) -> (heads, n, hd)
        return M.reshape(n, n_heads, hd).transpose(1, 0, 2)
    Q, K, V = split(X @ Wq), split(X @ Wk), split(X @ Wv)
    scores = Q @ K.transpose(0, 2, 1) / np.sqrt(hd) # every head at once
    scores += np.triu(np.full((n, n), -np.inf), k=1) # causal mask
    w = np.exp(scores - scores.max(-1, keepdims=True))
    w /= w.sum(-1, keepdims=True) # softmax
    out = (w @ V).transpose(1, 0, 2).reshape(n, d) # merge the heads
    return out @ Wo # output projection

rng = np.random.default_rng(0)
n, d = 3, 4
X = rng.normal(size=(n, d))
Wq, Wk, Wv = rng.normal(size=(3, d, d))
Wo = np.eye(d) # "do nothing" projection

one = multi_head_attention(X, Wq, Wk, Wv, Wo, n_heads=1)
two = multi_head_attention(X, Wq, Wk, Wv, Wo, n_heads=2)
print(one.shape, two.shape)
print(np.allclose(one, attention(X, Wq, Wk, Wv)))
print(np.allclose(one, two))
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. Several heads, side by side · rewatch ▶ 0:08-0:29

1.1 B. All heads read the same token vectors. Their own matrices let them ask different questions.

1.2 True. One head = one set of weights per token = one kind of question. Several heads give several rows, so several different mixes.

1.3 dog; quickly. Like *cat* and *slowly* for *sat* in the video: two questions, two different answers, so two heads.

2. Sharing out the vector · rewatch ▶ 0:29-0:40

2.1 64. $1,024 \div 16 = 64$.

2.2 8. $512 \div 64 = 8$.

2.3 False. $X \cdot W_Q$ uses all 768 numbers; it is the resulting query (and key and value) that is cut into 64-number pieces. Every head's 64 numbers are computed from the whole vector.

2.4 100; 1,200. Each head makes its own 10×10 grid (episode 6), and $12 \times 100 = 1,200$.

3. Different heads, different patterns · rewatch ▶ 0:40-1:10

3.1 B. Token i looks at token $i - 1$. D is impossible: the causal mask keeps everything above the diagonal empty.

3.2 False. The patterns are learned in training. Researchers find them afterwards, and many heads have no clear job at all.

3.3 Bridge. “the Golden Gate” appeared before, followed by *Bridge*. An induction head finds what followed the earlier copy and predicts that the phrase continues the same way.

4. Glue, mix and add · rewatch ▶ 1:10-1:27

4.1 project and split → every head runs attention → concatenate → multiply by W_O → add. Split before the heads work, glue after; W_O mixes the glued result, which is then added.

4.2 589,824. It is a 768×768 matrix: $768 \times 768 = 589,824$.

4.3 A. After gluing, each block of 64 numbers holds one head's findings; W_O lets them combine. B and C happen inside every head, D before.

4.4 False. The result is **added** to the token's vector (“residual: add, don't replace”), so the token keeps what it had and gains context.

5. Many views for the price of one · rewatch ▶ 1:27-1:37

5.1 1,769,472. $3 \times 768 \times 768 = 3 \times 589,824 = 1,769,472$.

5.2 49,152; 589,824. $768 \times 64 = 49,152$, and $12 \times 49,152 = 589,824$: exactly one 768×768 matrix.

5.3 False. The head size halves to $768 \div 24 = 32$, and $24 \times 768 \times 32 = 768 \times 768$ again.

5.4 A. Same cost, but each token can now look in twelve different ways at once. B and C are false, and D is still forbidden by the mask.

6. Multi-head attention in code · rewatch ► 1:37-1:57

6.1 (a) (12, 7, 64). (b) (12, 7, 7). (c) (7, 768). (heads, n, hd); one $n \times n$ grid per head; merging glues the heads back into one 768-number vector per token.

6.2 True. That is the outro's picture: every layer is attention, then MLP.

6.3 (a) $hd = 2$, Q (2, 3, 2), scores (2, 3, 3) · (b) (3, 4) (3, 4), then True, then False (a) $hd = 4 // 2 = 2$; `split` gives (heads, n, hd) = (2, 3, 2), and each head has its own 3×3 grid. (b) Both outputs keep X's shape, (3, 4), so they can be added back to X. (c) With one head, $hd = d$, `split` changes nothing, the scaling is $\sqrt{4}$ as in episode 6, and $W_O = \text{identity}$ changes nothing: it *is* single-head attention (True). With two heads, each head scores with only its 2-number half of the query and key (and divides by $\sqrt{2}$), so each gets its own weights and mixes its half of the values in its own way (False).